

B5: Sync

This lesson covers how applications keep their own copies of permissioned data current: incremental sync over the operation log, full-state recovery when incremental sync can't proceed, how blobs are fetched, how write notifications keep syncers from polling, and how a syncer discovers which repos make up a space in the first place.

The sync model: pull, with no relay

From B2 and B4: a space does not colocate records on any single host. Each member stores their own records for the space in a **permissioned repo** on their own repo host, and the space is the aggregation of those per-user repos across the network. An application that presents a space has to assemble it from many hosts, and unlike public atproto, no intermediary does the collation for it.

Permissioned data sync is functionally similar to public atproto. Applications build views by pulling repos from their hosts. The major difference is that there is no relay to provide a collated firehose of data for the network as permissioned repositories are by their nature non-rebroadcastable. An application pulls directly from each repo host and is responsible for keeping its own copy in sync.

Permissioned Data proposal, "Sync"

The proposal's terminology section names the actor doing this work:

Syncer: an application that keeps its own copy of a space in sync by pulling from repo hosts.

Permissioned Data proposal, "Terminology"

The division of labor drives everything in this lesson. Repo hosts (usually each member's PDS) store and serve the actual record data; every sync method that returns records is served by a repo host. The space host, resolved from the space authority's DID document, answers for the space as a whole: it issues credentials, enumerates writers, and routes notifications, but it does not store or stream the members' record data. The syncer pulls from each repo host, maintains its own copy, and owns the job of keeping that copy correct.

Pulling from every PDS sounds heavier than it is. The sync protocol is lighter-weight than public sync because there are no MST structural nodes to maintain and transmit. Sync load scales w. [?] the number of applications syncing a space rather than the number of end users: an application pulls

each repo once and fans it out to its users from its own copy. And the sync APIs are gated by space credentials rather than open to the public, so abuse is easier to rate-limit and control.

Incremental sync & the oplog

The set hash is the ground truth

Sync correctness in permissioned data does not rest on receiving every event. It rests on a digest comparison. From B4, each permissioned repo is summarized by a signed commit whose **hash** is **sha256** of the repo's LtHash set-hash state: a digest over the set of **{collection}/{rkey}/{record_cid}** elements the repo currently contains, independent of write order. The syncer maintains the same construction locally:

A syncer keeps its own copy of a repo and, alongside it, its own running set hash over that copy. This is the same digest that the repo host maintains. When the two digests agree, the syncer knows that their copy is exactly up-to-date with the hosted repo. If there is a disagreement, then the syncer knows their copy is behind or corrupted.

Permissioned Data proposal, "Sync → Incremental sync"

Matching revisions won't establish this. Neither will matching record counts, or having read the oplog to its head. Only agreement between the two set-hash digests establishes that the copies hold exactly the same records.

Advancing with listRepoOps

To advance its copy, a syncer calls **com.atproto.space.listRepoOps** on the repo host with a **since** revision. The repo host keeps an **operation log** (oplog) of recent writes to each repo and returns the operations after **since**. Each entry has the shape:

```
{ rev (TID), collection, rkey, cid, prev }
```

Each entry is **{ rev (TID), collection, rkey, cid, prev }**. The field **cid** is null for a delete and **prev** is null for a create. Multiple records may be mutated atomically, and this is captured by entries sharing a **rev**. The syncer applies each operation to its copy and updates its running set hash accordingly.

Permissioned Data proposal, "Sync → Incremental sync"

Field	Meaning
rev	The commit revision (a TID) at which this operation landed. Entries produced by one atomic batch (e.g. a single com.atproto.space.applyWrites call touching several records) all share one rev ; atomicity is expressed by the shared revision, not by merged entries or consecutive distinct revisions.

<code>collection, rkey</code>	The record path the operation touched. One entry describes one record path.
<code>cid</code>	The record's CID after the operation. Null for a delete (the path no longer has a value).
<code>prev</code>	The record's CID before the operation. Null for a create (the path had no prior value).

An update has both fields populated: `prev` is the old CID and `cid` the new one. As the syncer applies each operation it updates its running set hash: remove the `{collection}/{rkey}/{prev}` element if `prev` is non-null, add the `{collection}/{rkey}/{cid}` element if `cid` is non-null.

Inlined values and `excludeValues`

The entries themselves are pure metadata, but the default response carries more:

By default `listRepoOps` inlines each created or updated record's value alongside its operation, so a syncer advances in a single call without a `getRecord` round-trip per write. Only the current value for a path is inlined. If a record has been updated or deleted by a later operation, the host omits the intermediate (now stale) values. A syncer may set `excludeValues` to receive only the operation metadata (for example, to drive selective `getRecord` calls).

Permissioned Data proposal, "Sync → Incremental sync"

So the default response is self-sufficient for record data: metadata plus the current values, with stale intermediate values elided. `excludeValues` is the opt-out for syncers that want metadata only.

Checking the result against the commit

The syncer gets to verify its work whenever a response reaches the end of the log:

If a given oplog response includes the last available operation that occurred to a repo, then the response must also include the member's current signed commit. The syncer compares the commit's `hash` against its own running set hash. If they match, the syncer is fully in sync and the commit's signature authenticates that state. If they do not, the syncer has diverged and must fall back to a recovery process.

Permissioned Data proposal, "Sync → Incremental sync"

A mismatch is not symmetric. The commit is signed by the member and its MAC binds the hash to the commit context, so the host's state is authenticated; a digest disagreement means the syncer's copy is behind or corrupted, not that the commit is bad. The remedy is recovery: not rejecting the commit, not re-applying the same operations, and not waiting for a notification to deliver the missing data (notifications carry no data, covered below).

Sync is self-healing because its correctness rests on the set hash comparison rather than on receiving every operation. A missed operation is detected on a subsequent sync. A total

disjunction in repository history is also recognizable, and is recovered from by syncing the repository contents in their entirety.

Permissioned Data proposal, "Sync → Incremental sync"

The oplog is a transport optimization

Because correctness lives in the digest comparison, the oplog itself needs no durability guarantees, and the proposal gives it none:

The oplog is a transport optimization rather than a committed data structure. Its contents and history are not guaranteed, and a repo host may compact or drop it, retaining only a backfill window. It is also reset on account migration, as a new repo host begins its oplog afresh and does not inherit the prior host's history. In any such case, a syncer that cannot find its **since** revision falls back to full-state recovery, which does not depend on the oplog.

Permissioned Data proposal, "Sync → Incremental sync"

For a syncer this cashes out simply. The oplog is never a complete history from the repo's creation; it covers recent writes within whatever backfill window the host retains. A host that compacted or dropped its oplog has violated nothing, so a missing **since** revision is not a protocol error and not data loss. After an account migrates to a new PDS, the new host's oplog starts fresh; a **since** revision saved against the old host will not be found, and no backfill of oplog history from the previous host is coming. In every one of these cases the answer is the same: fall back to full-state recovery.

Full-state recovery

When incremental sync cannot proceed (the **since** revision is gone, the digests disagree after applying everything, or the syncer is starting from nothing), the syncer recovers the repo's full state.

To do so, a syncer fetches the whole repo as a serialized CAR from **com.atproto.space.getRepo**. It folds the index into a running set hash and compares that against the signed commit to authenticate the index, then validates each record block against its index CID as it streams past, rebuilding its local copy. A syncer replacing an existing copy diffs the recovered structure against what it holds and keeps only the records it is missing.

Permissioned Data proposal, "Sync → Full-state recovery"

This leans on the repo serialization from B4: the CAR's two roots are the signed commit and the index (a map from **"{collection}/{rkey}"** to record CID in lexicographic order), and the record blocks follow in the same order. The stream-validation procedure the serialization was designed for is exactly what the syncer runs here:

1. Verify the commit's signature and MAC. The commit's ``hash`` is now trusted.
2. Fold each index entry's ``{collection}/{rkey}/{record_cid}`` into a running set hash as it is read, then compare the result against the commit's ``hash``. This authenticates the whole index without reading a single record.
3. Verify each record block against its own CID as it streams past.

(Steps as given in the proposal's "Repo serialization" section, which `getRepo` serves.)

The lighter healing path

Full-state recovery always works, but it transfers the entire repo. For a copy that has only slightly diverged, the proposal offers a cheaper route:

For the narrower case of *healing* a copy that has only slightly diverged, a syncer may prefer to avoid transferring the whole repo. It can fetch the latest commit through `com.atproto.space.getLatestCommit`, enumerate the repo's structure (paths → CIDs) with `com.atproto.space.listRecords` using `excludeValues`, diff that lightweight listing against its local copy, and fetch just the differing records with `com.atproto.space.getRecord`. This trades the single `getRepo` round-trip for a smaller total transfer when most of the repo is already held.

Permissioned Data proposal, "Sync → Full-state recovery"

Four calls instead of one bulk download: `getLatestCommit` for the authoritative digest, a metadata-only `listRecords` for the path→CID structure, a local diff, then targeted `getRecord` fetches for the records that differ. All of these are repo-host methods. The syncer heals against the host that owns the repo; the space host holds no record data to heal from.

Blob sync

Neither `listRepoOps` nor the `getRepo` CAR carries blobs. The proposal's repo-serialization section states that "Blobs are not included and are fetched separately via `getBlob`", and the sync section is one sentence:

Blobs referenced by permissioned records are stored on the authoring repo's host and fetched via `com.atproto.space.getBlob` with the relevant space credential.

Permissioned Data proposal, "Sync → Blob sync"

Blobs live on the *authoring* repo's host, so you fetch a member's blobs from that member's host, addressed by CID. The fetch is authorized by the same space credential used for the rest of sync.

Write notifications

A syncer with no firehose still needs to know when to pull. It could poll every repo continuously; the protocol provides a push signal instead:

Write notifications inform syncers that a repo has advanced, so they can pull promptly instead of continuously polling for updates. A notification contains no record data, and states only that a given repo in a given space has reached a new revision.

Permissioned Data proposal, "Sync → Write notifications"

A notification is a doorbell, not a delivery. No record values, no oplog entries, no commit. Just "repo X in space Y has reached a new revision", after which the syncer pulls the updated repo directly from the relevant repo host, normally via `listRepoOps`.

Registering

A syncer subscribes by calling `com.atproto.space.registerNotify`, authenticated with a space credential. Where you call it decides what you get. Called on the **space host**, it subscribes to writes for all repos in the space; this is the general case, and syncers should subscribe to the space host for all of a space's write notifications. It can also be called on particular repo hosts to receive notifications for specific repos. The service registered against returns an expiration time for the registration, which may be longer than the expiration window of the space credential used to register.

The notification route

When a member writes, their PDS sends a `com.atproto.space.notifyWrite` to each endpoint registered for that repo. But a random member's PDS has no idea which applications are syncing the space. The space authority closes that gap:

A PDS may not otherwise know which services are syncing the space, which is why the **space authority** registers itself as a subscriber on each repo host. Members notify the authority, and the authority forwards each notification to the endpoints registered with it for the space. Each notified syncer then pulls the updated repo directly from the relevant repo host. The authority only routes notifications and does not carry record data.

Permissioned Data proposal, "Sync → Write notifications"

So the flow for a shared space is: member writes, the member's repo host notifies the space authority, the authority fans the notification out to the syncers registered with it, and each syncer pulls from the member's repo host. The authority is purely a router in this picture; the record data never passes through it.

The authority gets onto every member's repo host automatically:

A repo host does not need an explicit out-of-band registration step from the authority to know where to send these notifications. On the first write into a repo for a shared space (one whose authority is not the account's own DID), the repo host resolves the space authority's

`#atproto_space_host` endpoint and **auto-registers** it as a subscriber for that repo. The authority may later un-register if it no longer wishes to be notified. (Personal-data spaces, where the authority is the account's own DID and the PDS plays both roles, need no such registration.)

Permissioned Data proposal, "Sync → Write notifications"

Best-effort, by design

Notifications are **best-effort** and are not required for eventual consistency. If a notification is dropped, the affected repo is caught up by a later write's notification, or by a periodic sweep by the syncer. A sweep may be done over the writer set. `com.atproto.space.listRepos` returns each repo's current `rev`, so a syncer can compare those revisions against the revisions it last pulled and re-sync only the repos that have advanced, rather than polling each repo individually.

Permissioned Data proposal, "Sync → Write notifications"

This is the same design stance as the oplog: notifications are an optimization for promptness, never a correctness dependency. A dropped notification delays a pull. It can't cause permanent divergence, because a later write's notification or a periodic sweep catches the repo up. The sweep itself is cheap: one `listRepos` call to the space host yields every repo's current `rev`, and only repos whose `rev` has advanced past what the syncer last pulled need a re-sync.

The writer set

Everything so far assumed the syncer knows which repos exist. Syncing an individual repo requires knowing about it; syncing a space in full, or starting to sync a space for the first time, requires the set of accounts whose repos hold data in the space. That is the **writer set**, retrieved from the space authority via `com.atproto.space.listRepos`.

The authority is well placed to maintain it. Because it subscribes to updates from every repository in the space (the auto-registration above), it can keep a complete and current record of which repos hold data. Alongside each account, `listRepos` returns that repo's current `rev` and `hash`, which is what makes the periodic-sweep pattern possible. It's a simple fetch and carries no commit or history.

What the set enumerates is narrower than "the space's members":

It enumerates accounts that have **written at least one record** into the space, not the broader set of accounts that are merely *allowed* to write (which the authority may not even track) nor those that may only read it. Accounts that may read the space are never enumerated at the protocol-level, though applications may choose to enumerate them through records published in the space.

Permissioned Data proposal, "Sync → The sync boundary (writer set)"

And it is a claim, not a guarantee:

The writer set is what the authority *claims*, and is not itself authoritative for any individual repo. It is current as far as the authority's notifications have kept it. A listed account's repo host is the source of truth for whether that repo actually holds data. A syncer treats the writer set as the starting point for discovery and confirms each repo by syncing it directly from its host.

Permissioned Data proposal, "Sync → The sync boundary (writer set)"

So the writer set is a discovery aid. It excludes accounts that are merely allowed to write, it excludes readers, and it guarantees nothing about any individual repo. Bootstrapping a space comes down to: fetch the writer set from the space host, then sync each listed repo from its own host, treating each host as the source of truth for its repo.

The sync API surface, in one place

The methods this lesson used, as grouped in the proposal's XRPC API table. All live under `com.atproto.space`; the repo methods accept OAuth or a space credential, the host methods a space credential, and the notify procedures service auth.

Method	Role	Purpose in sync
<code>listRepoOps</code>	repo	Primary sync mechanism: the oplog since a given revision, record values inlined by default (<code>excludeValues</code> for metadata only).
<code>getRepo</code>	repo	Whole repo as a serialized CAR, for full-state backfill/recovery.
<code>getLatestCommit</code>	repo	The repo's current signed commit (start of the healing path).
<code>listRecords</code>	repo	List a repo's records; with <code>excludeValues</code> , a metadata-only paths→CIDs listing for diffing.
<code>getRecord</code>	repo	Fetch a single record's value (targeted fetches after a diff).
<code>getBlob</code>	repo	Fetch a blob by CID from the authoring repo's host.
<code>listRepos</code>	host	The writer set: known repos holding data in the space, with each repo's current <code>rev</code> and commit <code>hash</code> .
<code>registerNotify</code>	repo/host	Register an endpoint for write notifications: the whole space on the space host, a specific repo on a repo host.
<code>notifyWrite</code>	syncer/host	Notify that a repo advanced; sent by a repo host to the space host, forwarded by the space host to registered syncers.

You should now be able to explain

- Why there is no relay or collated firehose for permissioned data, and how an application obtains the data instead

- How a syncer knows its copy is exactly up-to-date: its own running set hash against the digest the repo host maintains, carried as the commit's **hash**
- The oplog entry shape **{ rev, collection, rkey, cid, prev }**, which field is null for a create versus a delete, and how an atomic batch appears (entries sharing a **rev**)
- What **listRepoOps** delivers by default (operations plus current record values, stale intermediates omitted), what **excludeValues** changes, and what it never delivers (blobs)
- Why a response that reaches the last available operation must include the current signed commit, and what a hash mismatch at that point means and requires
- Why the oplog is a transport optimization: no durability guarantee, compaction and backfill windows, reset on account migration, and the fallback when a **since** revision is not found
- Full-state recovery via **getRepo**: streaming CAR validation (commit, index fold, per-block CID checks) and diffing against an existing copy
- The lighter healing path for a slightly diverged copy: **getLatestCommit**, a metadata-only **listRecords** with **excludeValues**, a local diff, and targeted **getRecord** fetches
- How blobs are synced: stored on the authoring repo's host, fetched via **getBlob** with a space credential
- What a write notification does and does not contain, and how a syncer subscribes with **registerNotify** (space host for the whole space, repo hosts for specific repos)
- Why the space authority auto-registers as a subscriber on each member's repo host, and its role as a data-free notification router
- Why best-effort notifications don't threaten eventual consistency, and how a periodic sweep over **listRepos** revisions repairs dropped notifications
- What the writer set enumerates (accounts that have written at least one record, not allowed-writers, not readers), and why it is a claim to confirm against each repo host rather than an authoritative list